

CL3 Viva Theory & Code Walkthrough — DC + CI Edition

CL3 Viva Theory & Code Walkthrough — Detailed Edition

0. Foundations You'll Be Asked First

DC-1 — RPC: Distributed Factorial (Python xmlrpc)

Aim

Real-world analogy

Theory

Worked example — `client.factorial(5)`

Why Python (xmlrpc)?

Code walkthrough

Where used in real life

Likely viva questions

DC-2 — RMI: String Concatenation (Pyro5 + Name Server)

Aim

Real-world analogy

Theory

Worked example — `service.concat("hello", "world")`

Why Python (Pyro5)?

Code walkthrough

Where used in real life

Likely viva questions

DC-3 — Fuzzy Logic Operations

Aim

Real-world analogy

Theory

Worked example

Why Python?

Code walkthrough

Where used in real life

Likely viva questions

DC-4 — Load Balancing

Aim

Real-world analogy

Theory

Why Python?

Code walkthrough

Where used in real life

Likely viva questions

DC-5 — Genetic Algorithm for NN Hyperparameter Tuning

Aim

Real-world analogy

Theory

Why Python (DEAP + PyTorch + sklearn)?

Code walkthrough

Where used in real life

Likely viva questions

DC-6 — Clonal Selection Algorithm (CSA)

Aim

Real-world analogy

Theory

Worked example — 8-bit OneMax

Why Python?

Code walkthrough

Where used in real life

Likely viva questions

CI-1 — AIS for Structural Damage Classification

Aim

Real-world analogy

Theory

Worked example

Why Python?

Code walkthrough

Where used in real life

Likely viva questions

CI-2 — Distributed Evolutionary Algorithm with DEAP (OneMax)

Aim

Real-world analogy

Theory

Worked example

Why Python (DEAP)?

Code walkthrough

Where DEAP-style EAs are used in real life

Likely viva questions

CI-3 — MapReduce: Hottest / Coolest Year

Aim

Real-world analogy

Theory

Worked example

Why Python (without Hadoop)?

- Code walkthrough
- Where used in real life
- Likely viva questions

CI-4 — Ant Colony Optimization for TSP

- Aim
- Real-world analogy
- Theory
- Worked example — 4-city TSP
- Why Python (NumPy)?
- Code walkthrough
- Where used in real life
- Likely viva questions

Cross-Cutting Viva Questions

Glossary

Quick-Reference Cheat Sheet

CL3 Viva Theory & Code Walkthrough — Detailed Edition

A complete, *examples-first* guide to the 10 practicals of **417534: Computer Laboratory III**, organised by stream:

- **DC** — Distributed Computing (6 practicals)
- **CI** — Computational Intelligence (4 practicals)

For each practical you get:

1. **Aim** restated.
2. **Real-world analogy** to make the abstract idea concrete.
3. **Theory** — every concept and term explained in plain language.
4. **Worked example** — actual numbers and traces showing how the algorithm/protocol behaves.
5. **Why this language / framework?**
6. **Code walkthrough** — what each part of the code actually does and why.
7. **Real-world places this technology is used today.**
8. **Common viva questions, with detailed answers.**

A *cross-cutting* Q&A section with broader questions and a glossary appear at the end.

0. Foundations You'll Be Asked First

0.1 What is a “distributed system”?

A collection of independent computers connected by a network that **appears to the user as one coherent system**. They cooperate by passing messages.

Real-world analogy. A pizza delivery chain. The phone-line takes the order, the kitchen prepares the pizza, the rider delivers it. Three separate “machines” but to you it feels like one company. None of them shares memory — they coordinate by message passing.

Defining properties: - **No shared memory** — every node has its own RAM. Coordination is by message passing only. - **No global clock** — clocks drift; you cannot rely on absolute timestamps. - **Partial failure** — some nodes can fail while others keep running.

Goals: transparency, scalability, fault tolerance, resource sharing, concurrency.

0.2 Concurrency vs Parallelism vs Distribution

Term	Where?	Example
Concurrent	Multiple tasks make progress on one CPU (interleaved)	A waiter taking orders from many tables
Parallel	Multiple tasks at the same instant on multiple cores	Two waiters working two halves of the restaurant
Distributed	Tasks run on different machines via network	Chef in Pune, another in Mumbai, both serving Pune

0.3 Inter-process communication (IPC)

Mechanism	Where used	Example
Sockets (TCP/UDP)	Lowest level	Web browser → web server
RPC	Function-style remote calls	Practical DC-1
RMI / remote objects	Method-style remote calls	Practical DC-2
REST / HTTP	Web APIs	Twitter API
gRPC	Modern high-performance RPC	Google internal services
Message queues	Async, decoupled	RabbitMQ in food-delivery apps
Shared memory	Same machine	OS-level IPC
Pipes	Same machine	<code>ls grep .txt</code>

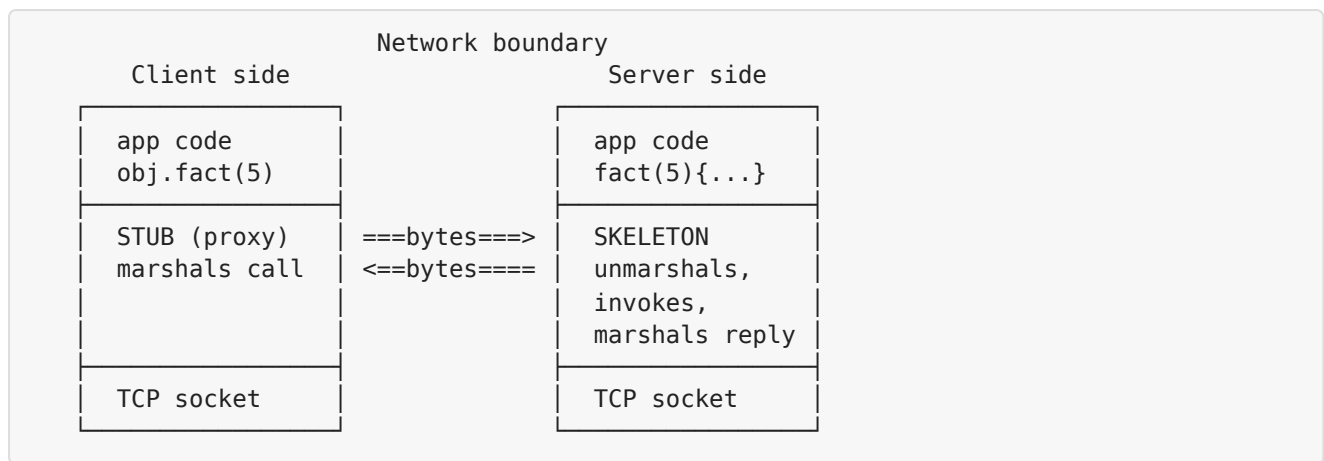
0.4 Synchronous vs Asynchronous

- **Synchronous** — caller blocks until reply (RPC, RMI). Simple to reason about.
- **Asynchronous** — caller continues; reply later via callback / future / event (message queues, async/await).

0.5 Marshalling / Serialization

Converting an in-memory object into a sequence of bytes that can travel over a network. Reverse is unmarshalling. RPC and remote-object frameworks rely on this.

0.6 Stub & skeleton



0.7 Failure semantics

- **At-most-once** — call executes 0 or 1 times. Most desirable; preserves correctness for non-idempotent operations.
 - **At-least-once** — may run multiple times due to retries. Fine only for idempotent ops.
 - **Exactly-once** — theoretical ideal; approximated with idempotency keys + dedup.
-

DC-1 — RPC: Distributed Factorial (Python xmlrpc)

Aim

A distributed application using **XML-RPC**: client sends an integer, server returns its factorial.

Real-world analogy

A restaurant. You don't have a kitchen at your table — you tell the waiter "I'd like pasta", the kitchen cooks it, the waiter brings it back. To you it looks like you "ordered pasta"; you don't see the kitchen. That's RPC.

Theory

What is RPC?

Remote Procedure Call lets your program call a function that runs *on another machine* as if it were local. The framework handles sockets, marshalling, transport and unmarshalling.

XML-RPC specifically

- A standard, language-neutral RPC protocol from 1998.
- The wire format is **XML over HTTP**: each call is an HTTP POST whose body is an XML document describing the method name and parameters.
- Reply is also XML: the return value or a `<fault>` element on error.
- **Built into Python** (`xmlrpc.server`, `xmlrpc.client`) — no extra installation.

Steps in an RPC call

1. Client calls stub: `result = client.factorial(5)`.
2. Stub marshals into XML: `<methodCall><methodName>factorial</methodName><params><param><value><int>5</int></value></param></params></methodCall>`.
3. HTTP POST sends the XML to the server.
4. Server unmarshals, calls the registered Python function `factorial(5)`.
5. Server marshals the return value `120` into XML and sends back.
6. Client unmarshals and returns `120`.

Factorial recap

$n! = 1 \times 2 \times 3 \times \dots \times n$, with $0! = 1$. Python integers are arbitrary precision, so even $100!$ (158 digits) works without overflow.

Worked example — `client.factorial(5)`

```
[Client] client.factorial(5)
|
v HTTP POST to http://localhost:8000/
| Body (simplified):
|   <methodCall>
|     <methodName>factorial</methodName>
|     <params><param><value><int>5</int></value></param></params>
|   </methodCall>
v
[Server] SimpleXMLRPCServer dispatches name "factorial" -> factorial(5)
factorial(5):
    f=1; for i in 1..5: f*=i -> 120
Marshals 120 -> XML response
|
v HTTP 200 OK
|   <methodResponse><params><param><value><int>120</int></value></param>
</params></methodResponse>
v
[Client] Receives 120, returns from client.factorial(5).
```

Why Python (xmlrpc)?

- **Standard library** — no `pip install` needed.
- Concise: server in 10 lines, client in 5.
- Cross-language interop: any XML-RPC-aware language (PHP, Java, C#) could call this server too.

Code walkthrough

`server.py`

```
from xmlrpc.server import SimpleXMLRPCServer
def factorial(n):
    if n < 0: return "Invalid input"
    f = 1
    for i in range(1, n+1): f *= i
    return f
server = SimpleXMLRPCServer(("localhost", 8000), allow_none=True)
server.register_function(factorial, "factorial")
server.serve_forever()
```

- `SimpleXMLRPCServer` is a tiny built-in HTTP server that speaks XML-RPC.
- `register_function(fn, name)` exposes `fn` under that name.
- `serve_forever()` blocks, dispatching requests in a loop.

client.py

```
import xmlrpc.client
client = xmlrpc.client.ServerProxy("http://localhost:8000/")
print(client.factorial(int(input("Enter: "))))
```

`ServerProxy` returns an object whose `__getattr__` builds RPC calls dynamically. `client.factorial(5)` becomes the HTTP POST shown above.

Where used in real life

- **WordPress.com** — XML-RPC API for blog posting.
- **Atom Publishing**, older blog/wiki integrations.
- **Pingback** in WordPress.
- More modern alternatives: **gRPC** (Google), **JSON-RPC**, REST.

Likely viva questions

Q: What is RPC? A: A protocol/paradigm where a program invokes a procedure on a remote machine as if it were local. The framework hides networking, marshalling, and error handling.

Q: What is XML-RPC? A: A specific RPC protocol that encodes method calls and responses in XML over HTTP. Standardised, language-neutral.

Q: What is `register_function`? A: Tells the XML-RPC server “expose this Python function under this name”. Clients call it by that name.

Q: How are arguments transmitted? A: Marshalled into XML elements (`<int>`, `<string>`, `<array>`, `<struct>`) and sent in the HTTP POST body.

Q: Synchronous or async? A: Synchronous. The client blocks on the HTTP response.

Q: Why not Pyro4/5 here? A: XML-RPC is in the Python standard library, language-neutral, and uses HTTP. Pyro is Pythonic only and uses its own binary protocol.

Q: What if the server crashes mid-call? A: The client gets a connection error / Fault. Decision to retry is the client's; for non-idempotent operations, retry must be done carefully.

DC-2 — RMI: String Concatenation (Pyro5 + Name Server)

Aim

Distributed application where the client invokes `concat(a, b)` on a remote object, located **by name** through a name server.

Real-world analogy

A library inter-loan. Your local library doesn't have the book; it consults the central catalogue (the **name server**) to find which branch holds it, then fetches it. You only ever talk to your local librarian.

Theory

What is RMI / a remote object?

RPC where you call **methods on remote objects** (stateful or stateless), not just standalone functions. Java's RMI is the classic implementation. Pyro (Python Remote Objects) is the Python equivalent.

Pyro5

- Successor to Pyro4. `pip install Pyro5`.
- Each remote object lives inside a **Daemon** that listens on a TCP socket.
- A **Name Server** maps friendly names (`"string.concat"`) to actual URIs (`PYRO:obj_xxx@host:port`).
- Default name-server port: **9090**. Default daemon port is randomly assigned.
- Default serializer is **serpent** (safe; no arbitrary code execution like raw pickle).

Name resolution flow

1. Client says `Pyro5.api.Proxy("PYRONAME:string.concat")`.
2. Proxy contacts the name server: "Where is `string.concat`?"
3. Name server returns the URI.
4. Proxy connects to that URI.
5. Method calls forwarded.

This indirection means clients don't hard-code addresses — useful when servers can move or restart on different ports.

Worked example — `service.concat("hello", "world")`

1. Client constructs `Proxy("PYRONAME:string.concat")`
2. Proxy queries the Name Server (`localhost:9090`):
`lookup("string.concat") -> "PYRO:obj_8ab2@localhost:51234"`
3. Proxy connects TCP to `localhost:51234`
4. Client calls `service.concat("hello", "world")`
5. Stub serializes (with `serpent`) and sends:
`method:"concat" args:["hello","world"]`
6. Daemon decodes, looks up `StringService` instance, calls `concat()`
`return "hello" + "world" = "helloworld"`
7. Daemon serializes "helloworld" and sends back
8. Client gets "helloworld"

Why Python (Pyro5)?

- Built specifically for Python — passing rich Python objects “just works”.
- Name Server makes services discoverable without hard-coded URIs.
- Less ceremony than Java RMI (no interface compilation step).

Code walkthrough

`server.py`

```
@Pyro5.api.expose
class StringService:
    def concat(self, a, b):
        return a + b

daemon = Pyro5.server.Daemon()
ns = Pyro5.api.locate_ns()
uri = daemon.register(StringService)
ns.register("string.concat", uri)
daemon.requestLoop()
```

- `@expose` declares this class is remotely callable.
- `locate_ns()` finds the running name server.
- `register(uri, name)` binds a friendly name in the name server.
- `requestLoop()` blocks, dispatching incoming RPC.

`client.py`

```
service = Pyro5.api.Proxy("PYRONAME:string.concat")
result = service.concat(a, b)
```

`PYRONAME:` prefix tells the proxy to look up the name in the name server first.

Where used in real life

- Pyro is used in scientific Python pipelines (custom data-processing services).
- Same architecture appears in **Java RMI** (used by Java EE, JBoss).
- The “name service” pattern is everywhere: **DNS** for the internet, **Eureka** in Netflix microservices, **Consul** for service discovery.

Likely viva questions

Q: What does the name server do? A: Maps a logical name (e.g. `string.concat`) to the actual network URI of the remote object. Clients can look up services by name, decoupling them from hostnames/ports.

Q: What's the URI format? A: `PYRO:objectId@host:port`.

Q: Why is `@Pyro5.api.expose` needed? A: Pyro refuses to expose anything not explicitly decorated. A “deny by default” security model.

Q: How is this different from XML-RPC (DC-1)? A: Pyro is binary, Python-specific, supports rich Python types (lists, dicts, custom classes). XML-RPC is text-based XML over HTTP, language-neutral, limited types.

Q: Synchronous or async? A: Synchronous by default. Pyro5 also supports oneway calls and futures.

Q: What if the name server is down? A: `locate_ns()` raises `NamingError`. Either the client falls back to a hard-coded URI or retries.

Q: How is this similar to Java RMI? A: Both look up remote objects by name (RMI uses `rmiregistry` instead of Pyro's name server) and transparently dispatch method calls.

DC-3 — Fuzzy Logic Operations

Aim

Implement Union, Intersection, Complement, Difference, Cartesian Product, and Max-Min Composition on fuzzy sets and relations.

Real-world analogy

The temperature of bath water. A *crisp* view: water is either hot ($>40^\circ\text{C}$) or not. A *fuzzy* view: water has a degree of hot-ness in $[0,1]$ — 25°C is 0.0 hot, 35°C is 0.3 hot, 50°C is 1.0 hot.

Theory

Fuzzy set

A set where each element has a **degree of membership** $\mu(x) \in [0,1]$ rather than a binary in/out.

Standard operations

Operation	Definition
Union ($A \cup B$)	$\mu(x) = \max(\mu_A, \mu_B)$
Intersection ($A \cap B$)	$\mu(x) = \min(\mu_A, \mu_B)$
Complement A'	$\mu(x) = 1 - \mu_A$
Bounded difference ($A - B$)	$\mu(x) = \max(\mu_A - \mu_B, 0)$
Cartesian product $A \times B$	$\mu(a,b) = \min(\mu_A(a), \mu_B(b))$ — produces a fuzzy relation

Fuzzy relation

A fuzzy subset of $A \times B$; assigns a degree to every (a, b) pair. Output of Cartesian product.

Max-Min Composition ($R \circ S$)

Given relations R on $A \times B$ and S on $B \times C$, the composition $R \circ S$ is a relation on $A \times C$ with $\mu_{\{R \circ S\}}(a, c) = \max \text{ over } b \text{ of } \min(\mu_R(a, b), \mu_S(b, c))$

This generalises matrix multiplication where $+$ is replaced by $\max$ and $\times$ is replaced by $\min$.

Properties

- **Idempotency:** $A \cup A = A$, $A \cap A = A$.
- **Commutativity, associativity, distributivity.**
- **De Morgan's laws** hold.
- **Excluded middle FAILS:** $A \cup A'$ need not equal the universe (if $\mu=0.5$, $\max(0.5, 0.5) = 0.5$).

Worked example

```
A = {a:0.5, b:0.8, c:0.2}
B = {b:0.6, c:0.7, d:0.3}
C = {c:0.9, d:0.4}

A ∪ B = {a:0.5, b:0.8, c:0.7, d:0.3}      (max of pairs)
A ∩ B = {b:0.6, c:0.2}                    (min on common keys)
A'      = {a:0.5, b:0.2, c:0.8}            (1 - x)
A - B   = {a:0.5, b:0.2, c:0}              (max(A-B, 0) per key)

R = A × B = { ('a','b'):0.5, ('a','c'):0.5, ('a','d'):0.3,
               ('b','b'):0.6, ('b','c'):0.7, ('b','d'):0.3,
               ('c','b'):0.2, ('c','c'):0.2, ('c','d'):0.2 }

S = B × C = { ('b','c'):0.6, ('b','d'):0.4, ('c','c'):0.7,
               ('c','d'):0.4, ('d','c'):0.3, ('d','d'):0.3 }

R ∘ S (a,c): max over b of min(R(a,b), S(b,c))
  For (a,c): try b='b' min(0.5,0.6)=0.5; b='c' min(0.5,0.7)=0.5; b='d' min(0.3,0.3)=0.3
  -> max=0.5
  ...
Result: {('a','c'):0.5, ('a','d'):0.4, ('b','c'):0.7, ('b','d'):0.4, ('c','c'):0.2,
         ('c','d'):0.2}
```

Why Python?

Operations are dictionary comprehensions; Python expresses them almost mathematically. No performance concern; concept is what matters.

Code walkthrough

Each operation is a one-liner. The Cartesian product builds a dict keyed by `(a,b)` tuples; max-min composition iterates pairs of those tuples and combines them with min/max.

Where used in real life

- **Sony / LG / Whirlpool washing machines** — fuzzy rules adjust water level + spin speed by load size.

- **Subway brakes** in Sendai (1987, the classic case study).
- **Camera autofocus** (Canon, Nikon).
- **Anti-lock brakes**, AC controllers, rice cookers.
- **Medical decision systems** — “mild fever” is inherently fuzzy.

Likely viva questions

Q: What is a fuzzy set? A: A set where each element has a membership degree in $[0,1]$, not the binary in/out of crisp sets.

Q: Define union/intersection/complement. A: $\max$ / $\min$ / $1-x$.

Q: What is a fuzzy relation? A: A fuzzy subset of a Cartesian product, assigning a membership degree to every pair.

Q: Explain max-min composition. A: For relations R and S sharing a common middle dimension B , the composition $R \circ S(a, c) = \max \text{ over } b \text{ of } \min(R(a, b), S(b, c))$. It generalises Boolean relation composition.

Q: Why does the law of excluded middle fail? A: Because for $\mu_A=0.5$ we have $\mu_{A \vee A'} = \max(0.5, 0.5) = 0.5$, not 1.

Q: Real example? A: Air conditioner — “IF temperature is HIGH AND humidity is HIGH THEN cooling is FAST”. Membership of “HIGH” is fuzzy, allowing smooth control.

Q: Difference between probability and fuzzy? A: Probability is *likelihood of yes/no events*; fuzzy is *degree of belonging to a category*. A 0.7 probability of rain means 70% chance rain *happens* (binary outcome). A 0.7 membership in “hot” means a temperature *partially qualifies* as hot.

DC-4 — Load Balancing

Aim

Simulate distribution of client requests across multiple servers using Round Robin, Least Connections, and Random algorithms.

Real-world analogy

Bank teller windows. The queue manager directs the next customer to one of several tellers. Strategies: cycle in order (round robin), send to whoever has the shortest queue (least connections), or pick at random.

Theory

What is a load balancer?

A component that distributes incoming requests across many backend servers to prevent overload, scale horizontally, and provide failover.

Layer 4 vs Layer 7

- **L4 (transport)** — uses IP and port. Fast, opaque to application data.
- **L7 (application)** — inspects HTTP headers/URL/cookies. Allows path-based routing, SSL termination, sticky sessions; slower.

Algorithms (with worked examples)

Round Robin — 10 requests, 3 servers:

```
Req:    1 2 3 4 5 6 7 8 9 10
Server: 1 2 3 1 2 3 1 2 3  1
Counts: A=4, B=3, C=3
```

Least Connections — pick the server with the smallest current load. Adapts to non-uniform request durations.

Random — pick uniformly at random. Statistically even for many requests but uneven for small N.

Weighted Round Robin — bigger servers get more turns: weights A=3, B=1, C=1 → expanded list [A, A, A, B, C], then RR.

Health checks

The LB pings each server (TCP open or `GET /health`). Failed servers are removed; recovered ones re-added.

Sticky sessions

Once a client is routed to server X, future requests go to X (via cookie or IP hash). Trade-off: simpler in-memory session state, but uneven distribution and broken when X dies.

Real-world load balancers

Hardware: F5, Citrix. Software: **Nginx**, **HAProxy**, **Envoy**. Cloud: AWS ELB/ALB/NLB, GCP, Azure.

Why Python?

Simulating an algorithm needs only a list and a loop. Python expresses each algorithm in 5 lines. Real production LBs are written in C (Nginx) or C++ (Envoy).

Code walkthrough

- `Server` — has `server_id` and `load` counter; `process_request` increments, `finish_request` decrements.
- `Request` — just an ID for tracing.
- `LoadBalancer` — holds the server list and a round-robin cursor.
 - `round_robin(req)` — `servers[i % N]`, then bump the cursor.
 - `least_connections(req)` — `min(servers, key=lambda s: s.load)`.
 - `random_assign(req)` — `random.choice(servers)`.

Where used in real life

Every site you visit is fronted by a load balancer (Google, Amazon, Netflix, Instagram). Netflix routes 200M subscribers' streams; WhatsApp dispatches 100B messages/day.

Likely viva questions

Q: Why do we need load balancers? A: To prevent server overload, scale horizontally, provide failover, and present a single endpoint to clients.

Q: Compare Round Robin vs Least Connections. A: RR is stateless and simple; perfect when servers and request durations are uniform. Least Connections tracks active load and adapts to uneven request durations (long-lived connections).

Q: What's a sticky session? A: Routing all requests from a given client to the same server. Pros: simpler in-memory session state. Cons: uneven distribution, breaks when a server dies.

Q: What if the LB itself fails? A: Single point of failure! Mitigated by deploying multiple LBs behind DNS round robin or virtual IPs (keepalived / VRRP).

Q: How does AWS ELB do health checks? A: Periodic TCP/HTTP probes to a configured endpoint. Unhealthy targets are removed from rotation.

Q: Why might you choose Weighted RR? A: When servers have different capacities (e.g., one 32-core, others 8-core). Higher-weight servers get more requests.

DC-5 — Genetic Algorithm for NN Hyperparameter Tuning

Aim

Use a Genetic Algorithm (DEAP) to evolve hyperparameters of a small PyTorch neural network: hidden-layer size, learning rate, activation function.

Real-world analogy

Selective breeding of crops for yield. Each season: measure yield, pick the top performers as breeders, mate them (calves inherit traits from both parents), occasional random mutation. Over many seasons, average yield rises.

Theory

Evolutionary algorithm framework

```
Initialize random population
For each generation:
    Evaluate fitness
    Select parents
    Crossover -> children
    Mutate children
    Form new population (often with elitism)
Return best
```

DEAP (Distributed Evolutionary Algorithms in Python)

- `creator` — defines custom Fitness/Individual classes.
- `base.Toolbox` — register operators (select, mate, mutate, evaluate).
- Built-in algorithms: `eaSimple`, `eaMuPlusLambda`.
- Statistics, hall-of-fame, multiprocessing support.

Hyperparameter tuning

Manually picking neurons / learning rate / activation is slow and brittle. A GA explores the space automatically:

Gene	Type	Range
<code>num_neurons</code>	int	5..100
<code>learning_rate</code>	float	0.0001..0.01
<code>activation</code>	category	relu / tanh / sigmoid

Fitness = test-set MSE after training the network for 50 epochs. **Lower MSE = better (minimisation problem).**

Operators we register

- **Crossover:** `tools.cxTwoPoint` — mix two parents at two points.
- **Mutation:** custom — Gaussian noise on numeric genes; 20% chance to flip the activation.
- **Selection:** `tools.selTournament(tournsize=3)` — pick 3 random, the best wins.

Why “distributed”?

Fitness evaluations are independent — each individual’s network can be trained on a different CPU core. DEAP supports `multiprocessing.Pool` and SCOOP.

Why Python (DEAP + PyTorch + sklearn)?

DEAP is Python’s standard EC framework. PyTorch + sklearn give us neural networks and dataset splitting in a few lines. Whole pipeline in <100 lines.

Code walkthrough

- `SimpleNN` — one hidden-layer feed-forward net (3 → hidden → 1).
- `objective(individual, ...)` — train the net for 50 epochs, return test MSE.
- `make_individual` — random hyperparameter triple.
- `mutate` — Gaussian noise + occasional activation flip; clamp to valid ranges.
- `algorithms.eaSimple` — runs the standard generational EA.

Where used in real life

- **NASA** — antenna design (ST5 spacecraft) by GA.
- **Drug discovery** — evolve molecule structures.
- **Google AutoML** — neural architecture search by evolution.
- **Game AI** — evolve strategies (StarCraft, Pac-Man).
- **Boeing** — wing-shape optimisation.

Likely viva questions

Q: What is a genetic algorithm? A: A population-based optimisation heuristic mimicking natural selection: encode candidates as “chromosomes”, evaluate fitness, select parents, apply crossover and mutation across generations.

Q: Role of crossover vs mutation? A: Crossover combines existing good traits (exploitation); mutation introduces novelty (exploration). Both are needed.

Q: What is tournament selection? A: Pick k individuals at random; the best of them is selected. $O(1)$ per selection, easy to tune by changing k .

Q: Why use a GA for hyperparameter tuning? A: The search space mixes integers, reals, and categoricals — gradient methods can’t navigate it. GAs are derivative-free and handle mixed types naturally.

Q: What is elitism? A: Keeping the best k individuals unchanged each generation. Prevents loss of the best solution to bad luck in selection.

Q: How do you avoid premature convergence? A: Higher mutation rate, larger population, diversity-preserving operators (niching), random restarts.

Q: Why DEAP? A: It is the canonical Python EC library — pure Python, multiprocessing-friendly for distributed fitness evaluation, easy to extend.

DC-6 — Clonal Selection Algorithm (CSA)

Aim

Implement the Clonal Selection Algorithm (CLONALG) on binary strings: maximise the number of 1s in an N-bit string.

Real-world analogy

How vaccinations work. When the flu virus enters your body: 1. B-cells with matching receptors are *selected*. 2. They *clone* themselves rapidly (millions in days). 3. *Mutations* in the receptor improve the match (somatic hypermutation). 4. Best clones become memory cells — next time, you're immune.

CSA mimics this for optimisation.

Theory

Algorithm steps

1. Initialise a random population of antibodies (binary strings).
2. For each generation:
 - Evaluate **affinity** (fitness) of each.
 - **Select** the n best.
 - **Clone** them — better antibodies get more clones.
 - **Hypermutate** clones (random bit flips with low probability).
 - **Replace** the worst antibodies with random new ones (preserves diversity).
3. Return the best antibody found.

CSA vs GA

	CSA	GA
Inspiration	Immune system	Natural selection
Crossover	None (asexual)	Yes
Variation	Hypermutation only	Mutation + crossover
Strength	Maintains diversity, multimodal	Faster on unimodal

Affinity

Problem-specific. Here we maximise the count of 1s in the binary string, so `affinity = sum(bits)`.

Worked example — 8-bit OneMax

Optimum is `[1,1,1,1,1,1,1,1]` with fitness 8.

```
GEN 0 (random pop_size=10):
  [0,1,1,0,1,0,0,1] fitness=4
  [1,0,1,1,0,1,1,0] fitness=5
  ...

GEN 1: top 50% = the 5 best
      clone factor 2 -> 10 clones
      hypermutate (mutation_rate=0.2 per bit):
      After ranking:
        best = [1,1,1,0,1,1,1,1]    fitness=7

GEN 2: best = [1,1,1,1,1,1,1,1]    fitness=8  (FOUND!)
```

Most runs converge to the optimum within a handful of generations.

Why Python?

Pure algorithm, no networking, no big libraries. Lists and `random` are enough.

Code walkthrough

- `fitness = sum(bits)`.
- `generate_antibody(size)` — random binary list.
- `clone_antibodies(selected, clone_factor)` — replicates each selected antibody.
- `hypermutation(clones, mutation_rate)` — flip each bit with probability `mutation_rate`.
- `replace_worst(population, keep_size)` — diversification by injecting random new antibodies.
- `clonal_selection` — orchestrates the loop for `generations` iterations.

Where used in real life

Network intrusion detection, anomaly detection, function optimisation, scheduling, pattern recognition. CSA is particularly good for problems with many local optima.

Likely viva questions

Q: What is CSA? A: An immune-inspired optimisation algorithm: select best antibodies, clone them, mutate clones, repeat.

Q: What is affinity? A: A problem-specific match score between antibody (candidate) and antigen (target). Higher = better.

Q: How is CSA different from GA? A: No crossover — only selection + cloning + hypermutation. Better at maintaining diversity, often better on multimodal problems.

Q: What is hypermutation? A: A higher-than-normal mutation rate applied to clones to thoroughly explore the neighbourhood of high-affinity solutions.

Q: Why also replace some worst antibodies with random new ones? A: To inject diversity. Without it, the population can collapse onto a local optimum.

Q: Real applications? A: Network intrusion detection (signatures of normal traffic), spam filtering, function optimisation, scheduling.

CI-1 — AIS for Structural Damage Classification

Aim

Apply Artificial Immune System pattern recognition to classify structural sensor readings as damaged or normal.

Real-world analogy

Airport security profiling. Over years of seeing normal passengers, security staff develop a sense of what's *normal*; anything outside that pattern is flagged. That's negative selection: learn what self looks like; treat non-self as anomaly.

Theory

Artificial Immune Systems (AIS)

Computational models inspired by the immune system. Used for: - Pattern recognition - Anomaly detection - Classification - Optimisation

Key AIS algorithms

Algorithm	Idea
Negative Selection	Generate detectors that DON'T match self; they then recognise non-self (anomalies).
Clonal Selection	DC-6
Immune Network	Antibodies stimulate/suppress each other
Danger Theory	Use "danger signals" rather than self/non-self

AIS vocabulary mapped to ML

AIS	ML equivalent
Antigen	Input pattern
Antibody / Detector	Model parameter
Affinity	Similarity (1/distance)
Self	Normal class
Non-self	Anomaly
Memory cells	Trained classifier

Distance / similarity metrics

- Euclidean (real-valued)
- Manhattan (grid)
- Hamming (binary)
- Cosine (high-dimensional)

How our classifier works

1. Sample some `num_detectors` points from the NORMAL class.
2. Mutate them (small Gaussian noise).
3. To classify a new sample x : compute distances from x to every detector. If the *minimum* distance is *greater than* the *mean* distance, the sample is anomalously far → label DAMAGED. Otherwise NORMAL.

This is a quirky variant — most negative-selection systems use a fixed threshold instead of comparing min-vs-mean — but it captures the anomaly-detection spirit.

Worked example

- 200 random samples in 5 dimensions.
- Label: `1` if `x[0] + x[1] > 1`, else `0`.
- 50 detectors drawn from class 0, mutated with Gaussian noise ($\sigma=0.05$).
- Test accuracy varies (~ 0.5 with the heuristic above; tune `num_detectors` and `mutation_rate` for better results).

Why Python?

NumPy makes vectorised distance computations trivial. scikit-learn provides `train_test_split` and `accuracy_score`.

Code walkthrough

```

class ArtificialImmuneClassifier:
    def generate_detectors(self, X_train, y_train):
        normal_samples = X_train[y_train == 0]
        idx = np.random.choice(len(normal_samples), self.num_detectors, replace=False)
        self.detectors = normal_samples[idx].astype(float)

    def mutate_detectors(self):
        self.detectors += self.mutation_rate * np.random.randn(*self.detectors.shape)

    def classify(self, X):
        labels = []
        for sample in X:
            d = np.linalg.norm(self.detectors - sample, axis=1)
            labels.append(1 if d.min() > d.mean() else 0)
        return np.array(labels)

```

- `np.linalg.norm(... axis=1)` computes Euclidean distance from `sample` to every detector in one vectorised operation.
- The classification rule (`min > mean`) is the simple anomaly heuristic.

Where used in real life

- **Aircraft structural monitoring** — Boeing 787 has thousands of sensors with AIS-style classifiers.
- **Long-span bridge monitoring** (e.g. Akashi-Kaikyō, Japan).
- **Network intrusion detection systems** (Snort and friends).
- **Manufacturing QC** — flag defective products on the line.
- **Antivirus signatures** — evolved from AIS concepts.

Likely viva questions

Q: What is an Artificial Immune System? A: A family of bio-inspired algorithms based on the immune system, used for pattern recognition, classification, optimisation, and anomaly detection.

Q: What is negative selection? A: An AIS technique that generates detectors that do NOT match training (self) data; deployed detectors then identify anomalies.

Q: What's affinity in AIS? A: A similarity measure between an antigen (input) and an antibody (detector). High affinity = strong recognition.

Q: Why use AIS for damage classification? A: Damage data is rare; AIS is good at one-class anomaly detection and supports online learning.

Q: How is this different from k-NN? A: k-NN stores all training points and classifies by majority vote among the k nearest. AIS produces a *compressed* set of detectors via cloning/mutation, often with internal dynamics that adjust over time.

Q: How would you compute affinity for binary strings? A: Hamming distance, then convert to similarity: `1 - distance / length`.

CI-2 — Distributed Evolutionary Algorithm with DEAP (OneMax)

Aim

Use DEAP to evolve a 10-bit binary string toward all-ones — the classical OneMax problem.

Real-world analogy

Same as DC-5: selective breeding. Here the “trait” is just the count of 1s.

Theory

OneMax problem

Trivial benchmark for evolutionary algorithms: maximise `sum(individual)` where individual is a binary string of length n . Optimum is `[1, 1, ..., 1]` with fitness n .

Why is it useful?

Because it's so simple, it isolates whether your GA setup is working. If a GA can't solve OneMax it's broken.

DEAP setup we use

- `creator.FitnessMax(weights=(1.0,))` — maximisation problem.
- `creator.Individual(list)` — chromosome is a Python list.
- `tools.cxTwoPoint` — two-point crossover.
- `tools.mutFlipBit(indpb=0.2)` — flip each bit with probability 0.2.
- `tools.selTournament(tournsize=3)` — tournament selection.
- `algorithms.eaSimple(... cxpb=0.7, mutpb=0.2, ngen=20)` — built-in generational EA.

Algorithm parameters

Parameter	Value	Meaning
Population	50	Number of individuals
Generations	20	Iterations
cxbp	0.7	Probability of mating
mutpb	0.2	Probability of mutating
tournsize	3	Tournament size
Hall of Fame	1	Tracks best ever

Worked example

Typical run output:

```

gen   nevals   avg   max
0      50      5.3    8
1      34      6.7    8
...
20     36      9.6   10

Best: [1,1,1,1,1,1,1,1,1,1] fitness=10.0

```

Population's `avg` rises generation by generation; `max` reaches 10 within ~5 generations.

Why Python (DEAP)?

DEAP is the standard EC framework in Python. The whole experiment fits in 30 lines.

Code walkthrough

```

toolbox.register("attr_bool", random.randint, 0, 1)
toolbox.register("individual", tools.initRepeat, creator.Individual, toolbox.attr_bool,
                 n=10)
toolbox.register("population", tools.initRepeat, list, toolbox.individual)

toolbox.register("evaluate", lambda ind: (sum(ind),))
toolbox.register("mate", tools.cxTwoPoint)
toolbox.register("mutate", tools.mutFlipBit, indpb=0.2)
toolbox.register("select", tools.selTournament, tournsize=3)

algorithms.eaSimple(pop, toolbox, cxbp=0.7, mutpb=0.2, ngen=20, halloffame=hof,
                   verbose=True)

```

DEAP's "register" pattern lets you swap operators by changing one line — for instance, `tools.cxOnePoint` for single-point crossover.

Where DEAP-style EAs are used in real life

- Antenna design (NASA ST5).
- Game AI evolution.
- Hyperparameter search (used in Google AutoML).
- Engineering design (Boeing wing shapes).
- Drug discovery.

Likely viva questions

Q: Explain `eaSimple`. A: DEAP's reference generational evolutionary algorithm: select parents → vary (mate + mutate) → evaluate offspring → replace population. Repeats for `ngen` generations.

Q: What does `cxTwoPoint` do? A: Picks two random crossover points in the parents, swaps the segment between them.

Q: What's `mutFlipBit(indpb=0.2)` ? A: For each bit independently, flip it with probability 0.2.

Q: What's the Hall of Fame? A: A small archive of the best-ever individuals seen across all generations. Survives even if the current population loses them.

Q: What is OneMax? A: The classic GA benchmark: maximise the count of 1s in a binary string. Trivial but useful for verifying setup.

Q: Distribution? A: DEAP's evaluation step can use `multiprocessing.Pool` or SCOOP to parallelise fitness evaluation across cores/machines — making it a *distributed* evolutionary algorithm.

CI-3 — MapReduce: Hottest / Coolest Year

Aim

Use the **MapReduce paradigm** to find the hottest and coolest year in a weather dataset, **without Hadoop** — pure Python with parallel mappers.

Real-world analogy

A national election count. - **Map**: each polling booth tallies its own votes. - **Shuffle**: results are sent to central counters by party. - **Reduce**: each central counter sums the per-booth tallies.

We do exactly this for temperatures by year.

Theory

MapReduce model

```
map(k1, v1)           -> list of (k2, v2)
reduce(k2, list of v2) -> list of (k3, v3)
```

Between map and reduce, the framework groups all values by key (the “shuffle and sort” phase).

How we implement it without Hadoop

- Read the CSV into a list of rows.
- **Split** the list into chunks of fixed size.
- **Map (parallel)**: `ProcessPoolExecutor` runs `map_function` on each chunk in a separate Python process. Each process returns `{year -> [temperatures]}`.
- **Shuffle**: combine partial maps so each year’s full temperature list is in one place.
- **Reduce**: for each year, compute `max` and `min` of its temperatures.
- **Final**: pick the year with the highest max (hottest) and the year with the lowest min (coolest).

Why this is “MapReduce” without being Hadoop

Hadoop is one *implementation* of the MapReduce model. The model itself is the idea: parallel map + grouping + reduce. We use Python multiprocessing — same paradigm, no cluster.

ProcessPoolExecutor

- Part of `concurrent.futures` in the Python stdlib.
- Spawns separate Python processes (avoiding the GIL — the Global Interpreter Lock that prevents true parallelism in threading for CPU-bound work).
- `executor.map(fn, iterable)` distributes items across processes.

Worked example

Input CSV:

```
year, temperature
2000, 10.5
2000, 12.0
2001, 5.2
2002, 15.3
...
```

After **map**:

```
worker 1: {2000: [10.5, 12.0], 2001: [5.2]}
worker 2: {2002: [15.3], 2003: [10.0]}
```

After **shuffle + reduce**:

```
2000: {max:12.0, min:9.8}
2001: {max:8.1, min:5.2}
2002: {max:15.3, min:14.1}
2003: {max:11.5, min:7.0}
```

Final result:

```
Hottest year: 2002 with max 15.3°C
Coolest year: 2001 with min 5.2°C
```

Why Python (without Hadoop)?

- The MapReduce model is **just an idea** — Hadoop is heavy machinery for petabytes. For small/medium data, Python's stdlib gives you the same paradigm in 30 lines, no cluster.
- `ProcessPoolExecutor` provides true parallelism (multiple processes, each its own Python interpreter).

Code walkthrough

- `read_weather_data(path)` — `csv.DictReader` returns a list of dicts.
- `map_function(chunk)` — group temperatures by year using `defaultdict(list)`.
- `reduce_function(grouped)` — for each year compute `max` and `min`.

- `map_reduce(file_path, chunk_size)` — orchestrates: split → parallel map → shuffle → reduce → pick best.

Where used in real life

- The pattern itself is foundational at **Google** (the original paper that invented MapReduce), **Facebook** (Hive on Hadoop), **Netflix**, **eBay**.
- Modern alternatives: **Apache Spark** (in-memory, 10-100× faster), **Apache Flink** (streaming), **Beam** (unified batch+stream).
- Standalone Python tools: **Dask**, **Ray**, **PySpark** in local mode.

Likely viva questions

Q: What is MapReduce? A: A two-phase parallel data-processing model. Map produces (key, value) pairs from input records; the framework groups them by key; Reduce aggregates values per key.

Q: Walk me through finding the hottest year using MapReduce. A: Map: emit (year, temperature) for each row. Shuffle: framework collects all temperatures per year. Reduce: for each year output `max(temps)` and `min(temps)`. Final: pick the year with the largest max as hottest, and the year with the smallest min as coolest.

Q: Why is Hadoop not needed here? A: MapReduce is a *paradigm*, not a product. Hadoop implements it for petabyte-scale clusters. For small/medium data we can implement the same logic with Python multiprocessing — no cluster overhead.

Q: What does `ProcessPoolExecutor` give us that threading does not? A: True parallelism for CPU-bound work. Python threads share a Global Interpreter Lock (GIL), so they can't run Python bytecode in parallel. Processes have separate interpreters and so can run on multiple cores at once.

Q: What is shuffle and sort? A: The framework's automatic phase between Map and Reduce: group all values for each key together so the reducer sees them all at once.

Q: Time complexity? A: Reading is $O(N)$. Map is $O(N)$ total work distributed across P processes — wall-clock $O(N/P)$. Reduce is $O(\text{distinct_keys} \times \text{records_per_key})$.

Q: Difference between Hadoop MapReduce and Spark? A: Hadoop MR writes intermediate data to disk between map and reduce — fault-tolerant but slow. Spark keeps it in memory (RDDs/DataFrames) — 10-100× faster for iterative algorithms.

CI-4 — Ant Colony Optimization for TSP

Aim

Apply ACO to find a short tour visiting every city in the Travelling Salesman Problem.

Real-world analogy

Real ants finding sugar in your kitchen. You leave a sugar spill; minutes later there's a trail of ants going nest → sugar → nest. How? Each ant lays a faint **pheromone trail** as it walks. Returning ants reinforce the trail. Shorter paths get more round-trips per minute → more pheromone → exponentially more attractive. Pheromone evaporates so weak trails fade. The colony converges on the shortest path with **no central planner**.

Theory

TSP

Visit every city exactly once and return to start; minimise total distance. **NP-hard** — brute force is $O(N!)$, so for $N > 20$ we need heuristics.

Swarm intelligence

Algorithms inspired by collective decentralised intelligence: ACO (ants), PSO (birds), Bee algorithms, Firefly algorithm.

ACO algorithm (Ant System for TSP)

For each iteration: 1. Place each ant on a random starting city. 2. Each ant builds a complete tour by probabilistic city selection: $P(j \mid \text{current}=i) \propto \tau(i,j)^\alpha \times \eta(i,j)^\beta$ where $\tau(i,j)$ = pheromone on edge (i,j) , $\eta(i,j) = 1 / \text{distance}(i,j)$ (heuristic), α weights pheromone, β weights heuristic. 3. After all ants finish, **evaporate** pheromone: $\tau \leftarrow \text{decay} \times \tau$ (decay < 1 in some formulations, our code uses `pheromones *= decay` with `decay=0.9` meaning 10% decay per iteration — keeping 90%). 4. **Deposit**: each ant adds $1/\text{length}$ to the pheromones on edges of its tour. Shorter tours deposit more. 5. Track the best tour seen so far across all iterations.

Parameter intuition

Parameter	Low	High
α	Ignore pheromone (random search)	Follow herd (premature convergence)
β	Ignore distance	Behave like nearest-neighbour
decay	Trails fade fast	Trails persist long

Typical: $\alpha=1$, $\beta=2$, decay around 0.5-0.9, ants $\approx N$, iterations 50-200.

Worked example — 4-city TSP

Distance matrix:

	A	B	C	D
A	0	10	15	20
B	10	0	35	25
C	15	35	0	30
D	20	25	30	0

Optimum tour by inspection: $A \rightarrow B \rightarrow D \rightarrow C \rightarrow A$ with length $10+25+30+15 = \mathbf{80}$.

Run with 10 ants, 50 iterations, decay=0.9, $\alpha=1$, $\beta=2$: the algorithm finds length 80 within the first iteration (small problem) and stays there — pheromones reinforce the optimal edges.

Why Python (NumPy)?

NumPy lets us express the pheromone matrix and pairwise distance computations as array ops, which is concise and fast for tutorial-sized problems.

Code walkthrough

`__init__`

Stores parameters and initialises `pheromones` to all-ones (uniform attractiveness).

`_select_next_city(current, visited)`

For each unvisited city, compute weight $\tau^\alpha \times (1/d)^\beta$. Normalise to a probability distribution. `np.random.choice(..., p=probabilities)` samples the next city.

`_construct_solution`

Each of `n_ants` builds a complete tour starting from a random city, then closes the loop by appending the start city. The total length is computed via summed distance.

`_update_pheromones`

- **Evaporation:** multiply the entire matrix by `decay` .
- **Deposit:** for each ant, add `1/length` to the pheromones of every edge it used.

`optimize`

Iterates `n_iterations` times, tracking the best tour found.

Where used in real life

- **UPS / FedEx route planning** — vehicle routing variants of TSP.
- **Network routing** — early TCP/IP papers using AntNet.
- **Telecom switching** — British Telecom used ACO for circuit routing.
- **Manufacturing scheduling** — job-shop problems.
- **Bioinformatics** — protein folding, DNA alignment.
- **Drone path planning**.

Likely viva questions

Q: What is TSP and why is it hard? A: Find the shortest tour visiting every city once and returning. NP-hard — no polynomial-time exact algorithm; brute force is $O(N!)$.

Q: How does ACO solve TSP? A: Ants build tours by probabilistic city selection guided by pheromone trails and inverse distance. Pheromone is reinforced on shorter tours and evaporates over time, so the colony converges on the best path.

Q: What are α and β ? A: α weights the pheromone (memory of past tours); β weights the heuristic (greed for short edges). Typical values: $\alpha=1$, $\beta=2$.

Q: Why pheromone evaporation? A: Without it, early random tours dominate forever. Evaporation lets the colony “forget” weak trails so the search continues to refine.

Q: ACO vs GA for TSP? A: ACO is constructive (builds tours edge-by-edge using probabilities); GA is generative (mutates/crosses whole tours). ACO often performs slightly better on routing-style problems.

Q: What is swarm intelligence? A: Collective problem-solving by decentralised agents following simple local rules. No central controller — global behaviour emerges from local interactions.

Q: Time complexity per iteration? A: $O(\text{ants} \times N^2)$ — each ant visits all N cities; each step considers up to N candidate next-cities.

Cross-Cutting Viva Questions

Q: Differentiate RPC and RMI. A: | | RPC | RMI | |---|---| | Paradigm | Procedural | Object-oriented | | Languages | Language-neutral | Java/Python frameworks | | Marshalling | XDR / XML / JSON / protobuf | Java / Pyro serializer | | Pass-by | Value | Value (Serializable), Reference (Remote) |

Q: Distinguish XML-RPC, Pyro5, Java RMI. A: XML-RPC is text-based XML over HTTP, language-neutral, limited types. Pyro5 is Python-only, binary, supports rich Python types via the serpent serializer. Java RMI is Java-only with native Java Object Serialization. All are synchronous remote-call frameworks.

Q: What does “distributed” mean across these practicals? A: Components running in separate processes (often on separate machines), coordinating via message passing. DC-1 and DC-2 are real network distribution. CI-3 distributes work across CPU cores via processes. The bio-inspired algorithms (DC-5, DC-6, CI-1, CI-2, CI-4) are conceptually distributable since fitness/affinity evaluations are independent and can run in parallel.

Q: Difference between fuzzy logic and probability? A: Probability is *likelihood of yes/no events*; fuzzy is *degree of belonging to vague categories*. Different mathematics, different intent.

Q: Why are bio-inspired algorithms (CSA, GA, ACO, AIS) useful? A: Many real problems are NP-hard, mixed-type, or non-differentiable. Classical exact methods fail. Bio-inspired heuristics are derivative-free, robust to noise, and find good (not provably optimal) solutions in reasonable time.

Q: Exploration vs exploitation? A: Exploration = wide search (high mutation, randomness). Exploitation = refinement of known good areas (greedy selection). Good optimisation needs both — start by exploring, gradually shift to exploiting.

Q: Common failure modes in distributed systems? A: Network partition, node crash, message loss, message duplication, message reordering, clock skew, byzantine faults.

Q: What is the CAP theorem? A: In a distributed data store you can guarantee at most two of: Consistency (all nodes see the same data), Availability (every request gets a response), Partition tolerance (system keeps working despite network splits). You always must choose P; the trade-off is C vs A.

Q: At-most-once vs exactly-once? A: At-most-once = call executes 0 or 1 times. Exactly-once is impossible under arbitrary failures but is approximated using idempotency keys + retries + server-side dedup.

Q: Synchronous vs asynchronous in practical terms? A: Sync: client blocks waiting (RPC, RMI, simple HTTP). Easy to reason about, slow under network latency. Async: client fires and continues (message queues, async/await). More efficient but harder to debug.

Glossary

Term	Meaning
Affinity	Similarity score in immune algorithms
Antibody	Detector / candidate solution in AIS
Antigen	Input pattern in AIS
At-most-once	Call executes 0 or 1 times
Cartesian product (fuzzy)	Pairwise relation built from two fuzzy sets
Convergence	Population narrowing around the optimum
Crossover	GA operator combining two parents
Crisp set	Classical Boolean set
Daemon	Long-running listener process
DEAP	Distributed Evolutionary Algorithms in Python
Elitism	Keeping best individuals across GA generations
Evaporation (ACO)	Decay of pheromone over time
Fitness	Score driving selection in EAs
Fuzzy set	Set with [0,1] membership values
Hypermutation	Higher-than-normal mutation rate (CSA)
Idempotent	Calling twice has same effect as once
Marshalling	Serializing objects to bytes
Max-Min Composition	Composition of two fuzzy relations using max-of-min
Membership function	$\mu(x) \in [0,1]$
MapReduce	Two-phase parallel data processing model
Name Server	Service mapping logical names to network URIs
NP-hard	Problem class with no known polynomial algorithm
Pheromone (ACO)	Reinforcement value on graph edges
Proxy / Stub	Client-side surrogate for a remote object
Pyro5	Python Remote Objects framework, current generation
Race condition	Bug where outcome depends on timing
RMI	Remote Method Invocation
Round Robin	Cyclic load-balancing algorithm
RPC	Remote Procedure Call
Selection (GA)	Choosing parents based on fitness
Serpent	Pyro's safe serializer
Shuffle and sort	MapReduce phase grouping values by key

Term	Meaning
Stub	Client-side proxy object
Swarm intelligence	Collective behaviour of simple agents
Synchronous	Caller blocks until result returns
Tournament selection	Pick k random, the best wins
TSP	Travelling Salesman Problem
URI	Uniform Resource Identifier (e.g., <code>PYR0:obj@host:port</code>)
XML-RPC	RPC protocol using XML over HTTP

Quick-Reference Cheat Sheet

Practical	One-line summary
DC-1	Client sends <code>n</code> , server returns <code>n!</code> over XML-RPC
DC-2	Pyro5 + Name Server: lookup <code>string.concat</code> by name, call <code>concat(a, b)</code>
DC-3	Fuzzy set ops + Cartesian product + max-min composition
DC-4	Round Robin / Least Connections / Random load balancing simulation
DC-5	DEAP GA evolves NN hyperparameters (neurons, lr, activation)
DC-6	CSA evolves a binary string toward all-ones via cloning + hypermutation
CI-1	AIS classifier — detectors trained on normal class, classify by distance heuristic
CI-2	DEAP solves OneMax (maximise count of 1s in 10-bit string)
CI-3	Python multiprocessing MapReduce finds hottest/coolest year
CI-4	ACO finds short TSP tour via pheromone trails

End of document. Read each practical's section once; re-skim Cross-Cutting Q&A and the cheat sheet just before the viva. Most examiners only have time for 4-6 questions, drawn predominantly from the per-practical "Likely viva questions" lists.